

Array Operations, Searching & Sorting

Class Teaching Notes — 2 Hour Session (120 minutes)

Follow-up topic to: 1D Arrays and 2D Arrays

Prerequisites (already covered)

- Declaring and initializing 1D arrays
- Declaring, initializing and traversing 2D arrays (matrices)
- Basic loops (for, while) and functions in C

Session Learning Objectives

- Pass arrays to functions correctly (1D and 2D)
- Perform core array operations: traversal, insertion, deletion, reversal, merging
- Implement and analyze **Linear Search** and **Binary Search**
- Implement and analyze **Bubble Sort** and **Selection Sort**
- Compare algorithms using Time Complexity (Big-O)

Lesson Plan / Timing

Time	Topic
0:00 – 0:10	Quick recap of 1D & 2D arrays
0:10 – 0:25	Passing arrays to functions
0:25 – 0:50	Array operations: traversal, insertion, deletion, reversal, merging
0:50 – 1:10	Linear Search
1:10 – 1:35	Binary Search
1:35 – 1:55	Bubble Sort & Selection Sort
1:55 – 2:00	Summary, complexity table, practice questions

0. Quick Recap — 1D and 2D Arrays (10 min)

Before moving ahead, revise the basics quickly on the board:

```
// 1D array
int arr[5] = {10, 20, 30, 40, 50};
for (int i = 0; i < 5; i++)
    printf("%d ", arr[i]);

// 2D array
int mat[2][3] = {{1, 2, 3}, {4, 5, 6}};
for (int i = 0; i < 2; i++) {
    for (int j = 0; j < 3; j++)
        printf("%d ", mat[i][j]);
    printf("\n");
}
```

Note: An array's name (e.g. **arr**) itself acts like a pointer to its first element — this is the key idea behind passing arrays to functions, covered next.

1. Passing Arrays to Functions (15 min)

Unlike normal variables, arrays are **not copied** when passed to a function — the function receives the **address of the first element**. So any change made inside the function reflects in the original array. This is different from passing a simple int or float, which is passed *by value*.

1.1 Passing a 1D Array

```
#include <stdio.h>

// Array size is passed separately since the function
// cannot know the size just from the pointer
void printArray(int a[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
}

void doubleValues(int a[], int n) {
    for (int i = 0; i < n; i++)
        a[i] = a[i] * 2;    // modifies the ORIGINAL array
}

int main() {
    int nums[5] = {1, 2, 3, 4, 5};
    printArray(nums, 5);    // 1 2 3 4 5
    doubleValues(nums, 5);
    printArray(nums, 5);    // 2 4 6 8 10 -> original changed!
    return 0;
}
```

1.2 Passing a 2D Array

For a 2D array, the **number of columns** must always be specified in the function parameter (the compiler needs it to calculate the memory offset of each row).

```
void printMatrix(int m[][3], int rows) {    // columns = 3 is compulsory
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < 3; j++)
            printf("%d ", m[i][j]);
        printf("\n");
    }
}
```

```
int main() {  
    int mat[2][3] = {{1,2,3}, {4,5,6}};  
    printMatrix(mat, 2);  
    return 0;  
}
```

Note: Common exam question: 'Why must array size be passed as a separate parameter?' — Because inside the function, **sizeof(a)** only gives the size of a pointer (usually 4 or 8 bytes), not the size of the whole array.

2. Core Array Operations (25 min)

These are the fundamental building blocks used in almost every array-based program. Walk through each with the array size vs. the number of elements currently stored (capacity vs. count).

2.1 Traversal

Visiting every element exactly once — the basis of printing, searching and summing.

```
void traverse(int a[], int n) {
    for (int i = 0; i < n; i++)
        printf("Index %d -> %d\n", i, a[i]);
}
```

2.2 Insertion (at a given position)

To insert a value at position **pos**, all elements from the end up to **pos** must be shifted one place to the right to create an empty slot.

```
// Inserts 'value' at index 'pos'. n = current number of elements.
// Array must have space for n+1 elements.
void insertAt(int a[], int *n, int pos, int value) {
    for (int i = *n; i > pos; i--)
        a[i] = a[i - 1];    // shift right
    a[pos] = value;
    (*n)++;                // one more element now
}

/* Example: a = {10, 20, 30, 40}, n = 4
   insertAt(a, &n, 1, 99)
   Step: shift 40->idx4, 30->idx3, 20->idx2
   Result: a = {10, 99, 20, 30, 40}, n = 5 */
```

2.3 Deletion (from a given position)

To delete the element at position **pos**, shift all elements after it one place to the left.

```
void deleteAt(int a[], int *n, int pos) {
    for (int i = pos; i < *n - 1; i++)
        a[i] = a[i + 1];    // shift left
    (*n)--;
}

/* Example: a = {10, 99, 20, 30, 40}, n = 5
   deleteAt(a, &n, 1) -> removes 99
   Result: a = {10, 20, 30, 40}, n = 4 */
```

2.4 Reversal

Swap elements from both ends moving towards the centre — the classic two-pointer technique.

```
void reverse(int a[], int n) {
    int start = 0, end = n - 1, temp;
    while (start < end) {
        temp = a[start];
        a[start] = a[end];
        a[end] = temp;
        start++;
        end--;
    }
}
```

2.5 Merging Two Sorted Arrays

A very common interview/exam problem — merge two already-sorted arrays into one sorted array without sorting again, using two pointers.

```
void mergeSorted(int a[], int n1, int b[], int n2, int result[]) {
    int i = 0, j = 0, k = 0;
    while (i < n1 && j < n2) {
        if (a[i] <= b[j])
            result[k++] = a[i++];
        else
            result[k++] = b[j++];
    }
    while (i < n1) result[k++] = a[i++];    // leftover elements of a
    while (j < n2) result[k++] = b[j++];    // leftover elements of b
}

/* a = {1,3,5}, b = {2,4,6}
   result = {1,2,3,4,5,6}    Time: O(n1 + n2) */
```

3. Searching Algorithms (35 min)

3.1 Linear Search (20 min)

Check every element one by one until the target is found or the array ends. Works on **sorted or unsorted** arrays.

```
int linearSearch(int a[], int n, int key) {
    for (int i = 0; i < n; i++) {
        if (a[i] == key)
            return i;        // found - return index
    }
    return -1;               // not found
}

int main() {
    int arr[] = {12, 45, 3, 67, 21};
    int result = linearSearch(arr, 5, 67);
    if (result != -1)
        printf("Found at index %d\n", result);
    else
        printf("Not found\n");
    return 0;
}
```

Case	Comparisons	Time Complexity
Best (found at index 0)	1	O(1)
Worst (not found / last element)	n	O(n)
Average	n/2	O(n)

3.2 Binary Search (15 min)

Much faster, but requires the array to be **already sorted**. Repeatedly divide the search range in half by comparing the key with the middle element.

```
int binarySearch(int a[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;    // avoids overflow
        if (a[mid] == key)
            return mid;
        else if (a[mid] < key)
            low = mid + 1;    // search right half
        else
            high = mid - 1;   // search left half
    }
    return -1;    // not found
}

int main() {
    int arr[] = {3, 8, 15, 21, 34, 45, 67}; // must be SORTED
    int result = binarySearch(arr, 7, 34);
    printf("%s\n", (result != -1) ? "Found" : "Not found");
    return 0;
}
```

Note: Trace this example on the board step-by-step: searching for 34 in {3,8,15,21,34,45,67} — low=0, high=6, mid=3 (value 21, 21<34 so low=4); low=4, high=6, mid=5 (value 45, 45>34 so high=4); low=4, high=4, mid=4 -> found! Only 3 comparisons instead of 5 with linear search.

Linear vs. Binary Search — Comparison

Feature	Linear Search	Binary Search
Array must be sorted?	No	Yes
Time Complexity	$O(n)$	$O(\log n)$
Approach	Sequential check	Divide and conquer
Good for	Small / unsorted data	Large sorted data

4. Sorting Algorithms (20 min)

4.1 Bubble Sort

Repeatedly compare **adjacent** elements and swap them if they are in the wrong order. After each full pass, the largest remaining element 'bubbles up' to its correct position at the end.

```
void bubbleSort(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int swapped = 0;
        for (int j = 0; j < n - i - 1; j++) {
            if (a[j] > a[j + 1]) {
                int temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
                swapped = 1;
            }
        }
        if (swapped == 0)    // already sorted, stop early
            break;
    }
}

/* Example: {5, 1, 4, 2, 8}
   Pass 1: 1 4 2 5 8
   Pass 2: 1 2 4 5 8    (already sorted, loop breaks) */
```

4.2 Selection Sort

In each pass, find the **minimum** element from the unsorted part and swap it with the first unsorted position.

```
void selectionSort(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (a[j] < a[minIndex])
                minIndex = j;
        }
        if (minIndex != i) {
            int temp = a[i];
            a[i] = a[minIndex];
            a[minIndex] = temp;
        }
    }
}

/* Example: {29, 10, 14, 37, 13}
   Pass 1: min=10 -> swap with a[0] -> 10 29 14 37 13
   Pass 2: min=13 -> swap with a[1] -> 10 13 14 37 29
   Pass 3: min=14 -> already in place -> 10 13 14 37 29
   Pass 4: min=29 -> swap with a[3] -> 10 13 14 29 37 */
```

Bubble Sort vs. Selection Sort

Feature	Bubble Sort	Selection Sort
Method	Swap adjacent out-of-order pairs	Pick minimum, place at front
Best Case	$O(n)$ (already sorted)	$O(n^2)$
Worst Case	$O(n^2)$	$O(n^2)$
Number of Swaps	Can be many	At most $n-1$

Stable?	Yes	No (in typical form)
---------	-----	----------------------

5. Summary — Master Complexity Table (5 min)

Algorithm	Best Case	Average Case	Worst Case	Space
Linear Search	$O(1)$	$O(n)$	$O(n)$	$O(1)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$

Key Takeaways to Emphasize

- Arrays are passed to functions **by reference** (via address) — changes persist outside the function.
- Insertion and deletion in arrays require **shifting** elements — $O(n)$ operation.
- Binary Search only works on a **sorted** array; it is much faster than Linear Search for large data.
- Bubble Sort and Selection Sort are both $O(n^2)$ but differ in *how* they compare/swap — good stepping stone before Insertion Sort, Merge Sort, Quick Sort in the next class.

6. Practice Problems (Homework / Lab)

- Write a function to insert an element into a sorted array so the array remains sorted.
- Modify Bubble Sort to sort in descending order.
- Implement Binary Search using **recursion** instead of a loop.
- Given an unsorted array, find the second largest element without fully sorting it.
- Merge two sorted 1D arrays of different sizes into a third array (already covered in class — attempt without looking at notes).
- Trace Selection Sort by hand for the array {64, 25, 12, 22, 11} and write down every pass.

Next class preview: Insertion Sort, Merge Sort, and an introduction to recursion-based array algorithms.